

Freek van den Berg s0517593

Business Rules: Final research paper

Topic: Formulating and managing Business Rules: can the business be expected to do it by themselves?

June 22nd, 2009

Introduction

Nowadays the term Business Rules receive a lot of attention when one talks about business and ICT. Business rules are a set of rules aiming to steer the business a certain way which matches the long term goals it is aiming for. Since most businesses drive for a major part on ICT, ICT people can not ignore the existence of Business Rules and are therefore forced to deal with them. The problem here is that Business Rules change more quickly than the ICT infrastructure. Because of this an extra layer is needed in the software to make a connection between the business and ICT. Also one would like the Business Rules to be executed by the ICT infrastructure automatically without having to have humans interfere all the time. This requires rules to be both formal and understandable by humans. It turns out that going from a formal language to a natural language is often still doable, but the other way around is very tricky. Business people are not programmers and are often not familiar with formal languages. This induces that it is very hard to have what is in business people's minds translated to formal rules, either directly or with a mediator. This paper will propose an philosophical approach in which rules are created in an interactive session between a business person and a computer. Instead of seeing a computer as a pain which only accepts formalized, mathematical statements, an approach in which the computer is some sort of assistant is chosen. The assumption here is that the computer is already familiar with the conceptual domain and its population. This leads to the following research question: Is it possible to design a system which a business person can use to interactively create formal business rules?

Literature study

The first step in answering the research question is doing a literature study. This literature study deals with four papers. One is about system theory to see what is actually needed from a system to survive in a complex world. After this ORM and SBVR are being dealt with for seeing what the business rules are about and how they could be expressed in a formal but readable way. Finally a system called Declare which has been developed in Eindhoven is looked at in order to see how rules are connected to each other and how they could be integrated in work flows.

Firstly there is Ashby's system theory[1]. Ashby states that when the management of a system is not complex enough to deal with the complexity of the environment (which is virtually always the case), essential variables of the system should be detected and kept within a certain span. For example, in order to keep a human alive, a body temperature between 35 and 42 degrees is highly wanted. To keep a value within a certain span, Ashby introduces regulatory actions when a disturbance (some unwanted influence from the environment) happens. To continue with the body temperature example, seeing a doctor would be a regulatory action for the disturbance getting a virus causing a high body temperature.

Secondly ORM[2], the object role modeling language by Terry Halpin. ORM provides a language in which meta models can be constructed in a formal way. This is done by defining concepts (object types) and the relations between them (fact types). The idea behind this started with NIAM, in which communication has been taken as starting point for describing something. This turned out to be a good approach since communication is the way humans describe something. After a meta model has been defined, it can be filled with a population. The just mentioned example about a person having a body temperature is expressed as follows: PERSON has body temperature BODYTEMPERATURE degrees. A population of this fact type could be {John has body temperature 37 degrees. Pete has body temperature 38 degrees}. If one wants the body temperature of the people in the population to always be between 35 and 42 degrees a constraint stating BODYTEMPERATURE is element of {35...42} can be introduced in ORM.

Thirdly SBVR[3], which stands for Semantics of Business Vocabulary and Business Rules and has been derived from ORM and therefore shows tight connections with it. The language has been designed for describing a complex entity in a formal way and is just like ORM based on communication. Two major aspects can be distinguished in the language which can also be combined, namely a fact oriented view or a rule based approach. The former is a lot like the ORM language and deals with describing object types, fact types, but also rule descriptions. The description of the rule as just mentioned about the body temperature can be translated as follows: The body temperature is between 35 and 42 degrees. Which is already more readable and closer to natural language. In addition to this the rule based approach offers statements about fact types. Examples are: It is possible that a body temperature is below 35 degrees. It is required that a body temperature is between 35 and 42 degrees.

Finally the Declare system is introduced by M. Pesic et al [4] in order to make it possible to define rule in a work flow kind of way. The approach is not completely like work flows though, because not every single step and transition needs to be defined. Instead of this an approach in which rules and temporal dependencies are defined per case is chosen. This approach can be translated to state transition machines. By using this approach in addition to ORM and SBVR, it becomes possible to not just express rules in terms of the state of the system, but also by defining something that is similar to work flows

Designing a system

The next step after the literature involves the design of a fictional system in a mostly philosophical way. The system does not necessarily be created in real, but can be used as a an example or thought experiment for where to go with Business Rules. When one wants to view this system from a business point of view, the word system can be changed by company where ever mentioned.

Ashby's system theory states the essential variables have to be defined. What can be derived from the continuation of the literature study in this paper is that these essential variables are chosen to be object types and fact types. Acquiring these object types and fact types is a very hard task which requires a domain expert and an information specialist. The assumption here is that this task has already been successfully completed and the focus shall be on creating rules when the domain is already known. It is important to mention here that selecting the object types and fact types already puts many constraints on the system. This happens when a domain is selected for the label types or when fact types are connected in a restricting way. An example here is: A marriage consisting of a man and a woman and a man and a woman being a subtype of the object type human. In more and more countries same sex marriage is allowed now too, which requires a change in the conceptual model to implement it. Another design decision could have been to make a marriage consist of two humans from the start and have a complex constraint state that the gender should be different. In the latter case the removal of the constraint would have been enough without having to change the conceptual model. Therefore it is important to have some sort of flexibility in the conceptual model. The assumption is that the conceptual model contains enough flexibility. Complete flexibility is however not needed. If one changes his business from selling apples to manufacturing cars, it should not be considered strange that a whole new conceptual model is needed.

After the conceptual model is there, the complete scope of the system is known. Since every object type and fact type is in the conceptual model for some reason, it makes sense to consider them all essential variables. The values these essential variables can take are defined by the label types in the model and the maximum size of the population per object type and fact type. Now is the time Business Rules are starting to play a role, because it is not possible to regulate the system just by setting the domains of the label types right because of the following reasons:

- A change in rules would require a change in the conceptual model. This can have big consequences such as a change in the database structure that is based on that conceptual model.
- In some cases some values would be allowed and in some cases not depending on the population of other object types. A domain only restricts the values of one object type, independent of the context.

In Ashby's system theory disturbances are defined as the essential values going out of their wanted reach. Translating this to the conceptual model, this means a population occurs that is not wanted. There are two ways to deal with this. Firstly one could not allow a transition that leads to this population and secondly it is possible to try to fix this population by having something changed right after the illegal population has been reached. In both cases this means that the illegal population has to be detected. This is done by checking all constraints after a transaction in the population has been done. These constraints are called Business Rules in this approach and the challenge here is to find the right Business Rules that make sure the system does not reach an illegal state. This requires getting the rules out of the business person's head and having them in the system in a formalized way, as quick and as efficient as possible. To achieve this one would ideally prefer to have the business person interact with the system directly, without an information specialist needed.

In the previous paragraph the word transaction is mentioned when checking the rules is the case. A transaction is a set of changes to the population that represent some sort of action in the real system. It is not possible to check all the constraints after every single change to the population, since sometimes different changes belong together. This makes it possible to have an illegal population during a transaction, while nothing illegal is happening. This is where the Declare system by M. Pesic comes in handy, because this system still makes it possible to check during a transaction (which is a case of a work flow) whether the transaction can still be finished in a satisfying way. This is done by checking all the constraints that are involved in the transaction and checking whether still a path exists in which they can all be satisfied. The downside of this approach is that only constraints that are case dependent can be checked and therefore this is not the only way this report is going.

Interacting with the system

This chapter is where the real work with respect to business rules happens. So far a system has been designed with a conceptual model in ORM/SBVR style. The model is supposed to be a translation of the real world of the system and therefore the only focus will be the population of the model from now on, regardless what happens in the real world (close world assumption). Let us assume that there is already a sample population, either fictional or non-fictional available to work with. The task now would be to have a business person interact with the system and set up a new business rule (which is a constraint to the population). Since the business person is possibly not be mathematically skilled and does perhaps not use formal languages at all, one can not expect the business person to just write down a formal statement which exactly represents what he means. Because of this there are three ways to go:

- A business person just clicks through different menus and buttons, while the system shows everything it knows about the conceptual domain and business rules.
- A business person types what he wants if he were talking to a human and the system can deal with it
- A business person types something close to what he means, the system does suggestions back, the business person changes what he had typed at first and so forth.

The first option has two major disadvantages. First of all this way is very inefficient, since the keyboard is not or not optimally used at all. Entering a business rules by clicking word by word can be very slow. Secondly the business person might not be able to find what he wants directly or does not know how to start. And finally when everything needs to be expressed in buttons and menus, the language might become not rich enough.

The second option requires highly sophisticated AI. First of all the system needs to understand natural language. Secondly even humans can not always perform the task of understanding natural language at once, simply because there might be ambiguities or errors in the language. And thirdly the user might not even be able to express or know exactly what he wants.

This leaves the third way, in which setting up a business rules is an interaction between a business person and the system.

What needs to be defined now is how these interactions take place and what they look like. Since there is information about the domain available and a sample population along with it, the system will already have something to work with, even when the user does not provide any information yet. To illustrate better what is going on, the following sample conceptual model with a population is used for illustration purposes

Student		Student	Course		Course
John	follows	John	Mathematics	is followed by	Mathematics
Pete		Pete	Physics		Physics
Allison		Allison	Physics		English

The just shown model has two object types (Student and Course) which are connected through a fact type (StudentCourse). For every object type and fact type the population consists of three entities.

The following possible interactions have been thought of which will each be explained carefully:

- A user does not provide input and the system comes up with business rules by performing data mining
- A user starts typing in (semi) natural language and the system tries to translate this to SBVR. This typing might include the use of buttons with predefined texts and the auto complete function which tries to finish words the user types.
- A user starts providing populations that are not wanted and the system shows different business rules that restrict that population, starting from the most general ones.

Firstly the data mining variant. Assume the above conceptual model and population is given to the system. What can be noticed in the StudentCourse table is that every student is following exactly one course. This might lead to the following suggestions for Business Rules by the system:

- it is obligatory that one student follows at least one course (total role constraint)
- it is obligatory that one student follows maximally one course (uniqueness constraint)

It is important to have a good ordering in the rules because also the following more specific rules match:

- it is obligatory that student John follows the course Mathematics
- it is forbidden that student Pete follows the course Mathematics

Possibly the data mining process can be guided using user input to for example have a scope.

Secondly the variant in which a user starts typing using a sophisticated user interface. Imagine a user typing:

- Every student has to follow one or more courses...

The system then might reply with different suggestions that are close to what the user means. After clicking on one and possibly editing that one, the system can come up with new suggestions again. This interactions continues until some satisfactory result has been reached.

The previous two steps can be supported by showing sample populations that are restricted by the newly defined rule and also show populations that are permitted. Both of those populations should be close to the Business rule's reach, so that the population illustrates well what the rule is doing to the boundaries of the system.

Thirdly the providing of illegal populations by the user. A user could decide in the above example to provide the following population to be illegal:

Student={John}, StudentCourse={(John,English),(John,Dutch)}, Course={Dutch,English}.

The system might conclude from this that a student is not allowed to follow two courses and come up with:

- it is obligatory that one student follows maximally one course (uniqueness constraint)

Once again many rules can forbid this population and therefore it is again important to show the most general ones first, since they are more likely the rules the business is looking for. As a measure for generality one can take the following rule of thumb: the length of the rule should be short and the size of all possible populations it restricts big.

Finally it is worth mentioning that a combination of all three methods can be very powerful. An example: A user can type something first, after which the system data mines for matches and after clicking on the different options an illegal population is shown. This illegal population can be edited by the user, after which new potential rules show up. This process can continue until satisfying results are reached.

Conclusion

This paper has shown an approach for Business Rules that can be taken in the future. The paper starts with a literature study, in which system theory leads to having to define essential variables, disturbances and regulatory actions. ORM and SBVR provide a solution here in terms of a conceptual model of which the population needs to be kept within certain boundaries. The rules, which can be defined in SBVR, provide regulatory actions here.

After this interactions with the system are described in which a user can make new Business rules in three ways. The system has the user input, the conceptual model and a sample population to work with to support in these processes. In the first way the system simply data mines for new rules, after which a user can select an appropriate one. Secondly the user can start typing in natural language, while the system tries to translate what the user is typing to SBVR, using the conceptual model. In the third case the user provides a population that is not wanted after which the system returns rules that forbid that population. And finally all the methods are combined in a short example.

References

- [1] W.R. Ashby, Principles of the self organizing system, Pergamom, 1962.
- [2] T.Halpin, ORM2, Springer Berlin / Heidelberg ISBN 978-3-540-29739-0, 2005
- [3] P. Bollen, SBVR: A Fact-Oriented OMG Standard, Springer Berlin / Heidelberg, 5333/2008
- [4] M Pesic, H Schonenberg, WMP van der Aalst, Declare: Full support for loosely-structured processes, 11th IEEE International Enterprise Distributed Object, 2007